Παράλληλη Επεξεργασία Διανυσματικών Συζεύξεων με LSH στο Apache Spark

Επεξεργασία Μεγάλων Δεδομένων

"Μεγάλα δεδομένα και αναλυτική"

Περίληψη

Στη παρούσα εργασία, γίνεται προσπάθεια υλοποίησης αποδοτικού αλγορίθμου, για τον υπολογισμό ζευγαριών, από ένα σύνολο διανυσματικών δεδομένων, των οποίων η ευκλείδεια απόσταση δεν υπερβαίνει μια παράμετρο θ, την οποία καθορίζει ο χρήστης. Η εύρεση και υλοποίηση του αλγόριθμου, καθίσταται αναγκαία, καθώς το παραπάνω πρόβλημα είναι υπολογιστικά ακριβό όταν αναζητείται μια ακριβής λύση. Συνεπώς, αναζητείται μια προσεγγιστική λύση, η οποία πλησιάζει αρκετά στην ακριβή, ωστόσο μπορεί να υπολογιστεί πολύ ταχύτερα και να είναι άμεσα κλιμακώσιμη. Η λύση η οποία προτείνεται είναι βασισμένη στον αλγόριθμο LSH, Locality - Sensitive Hashing και υλοποιείται στο Apache Spark, για παράλληλη εκτέλεση. Το σύνολο δεδομένων είναι το SIFT1M και στην πλειοψηφία των πειραμάτων επιτυγχάνεται Recall της τάξης 90-95%, όπως και απαιτείται.

CCS CONCEPTS

Παράλληλη επεξεργασία δεδομένων

Λέξεις κλειδιά

LSH, Distance Join, Apache Spark, Ευκλείδεια απόσταση, παράλληλη επεξεργασία, SIFT1M

Εισαγωγή

Το πρόβλημα της σύζευξης ζυγών διανυσμάτων, θα μπορούσε να θεωρηθεί ως ένα από τα πιο σημαντικά, στο πεδίο της επεξεργασίας μεγάλων δεδομένων. Δοθέντων δύο συνόλων διανυσμάτων, R, S τα οποία έχουν n διαστάσεις, ζητείτε ο υπολογισμός όλων των ζευγών, (r, s), $r \in R$, $s \in S$, των οποίων η ευκλείδεια απόσταση d(r,s) δεν υπερβαίνει μια παράμετρο θ, την οποία σε κάθε εκτέλεση ορίζει ο χρήστης. Η ακριβής λύση του προβλήματος, απαιτεί εξαντλητική σύγκριση κάθε ζευγαριού, με υπολογιστική πολυπλοκότητα $O(R \times S)$. Στην περίπτωση μας, η οποία αφορά σύνολο δεδομένων με εκατομμύρια διανύσματα, η ακριβής λύση καθίσταται αδύνατη. Για παράδειγμα, σε δοκιμή με 10.000 x 10.000 διανύσματα, ο baseline αλγόριθμος δεν κατάφερε να ολοκληρωθεί ούτε σε διάστημα μεγαλύτερο των 2 ωρών, αποδεικνύοντας έτσι τον παραπάνω ισχυρισμό. Για την αντιμετώπιση αυτού του υπολογιστικού προβλήματος, προτείνεται η χρήση του αλγορίθμου Locality-Sensitive Hashing (LSH) [1], ο οποίος αποφεύγει την εξαντλητική αναζήτηση κατευθύνοντας τη σύγκριση μόνο σε υποψήφια ζευγάρια που είναι πιθανώς κοντινά. Ο αλγόριθμος υλοποιείται στο Apache Spark, σε περιβάλλον Google Colab, προσομοιώνοντας την παράλληλη επεξεργασία για κλιμακώσιμη εκτέλεση σε μεγάλα σύνολα δεδομένων.

Θεωρητικό Υπόβαθρο

Το Locality-Sensitive Hashing (LSH) είναι μια προσεγγιστική τεχνική κατακερματισμού που χρησιμοποιείται για την αποδοτική αναζήτηση ομοιότητας σε χώρους υψηλών διαστάσεων. Η βασική ιδέα είναι η χρήση hash functions που τοποθετούν κοντινά διανύσματα στο ίδιο bucket με υψηλή πιθανότητα, ενώ απομακρυσμένα διανύσματα τοποθετούνται σε διαφορετικά buckets. Για το πρόβλημα της Ευκλείδειας απόστασης, αξιοποιούντα hash functions, τα οποία προτείνονται ως λύση από τον Datar et al. (2004) [2]. Κάθε hash function ορίζεται ως:

$$h(x) = \lfloor (a \cdot x + b)/w \rfloor$$

Όπου a ορίζεται ως ένα τυχαίο διάνυσμα με τιμές από κανονική κατανομή $N(0,1)$, b είναι ένας τυχαίος αριθμός ομοιόμορφα κατανεμημένος στο διάστημα $[0, W]$ και w είναι το πλάτος του bucket. Το εσωτερικό γινόμενο $a \times x$ αποτελεί την προβολή του διανύσματος x πάνω στην τυχαία κατεύθυνση a . Διανύσματα που βρίσκονται κοντά στον 128-διάστατο χώρο τείνουν να έχουν παρόμοιες προβολές και κατά συνέπεια να ανατίθενται στο ίδιο bucket.

Ο αλγόριθμος χρησιμοποιεί L ανεξάρτητα hash tables, καθένα από τα οποία αποτελείται από k hash functions. Για κάθε table, τα k αποτελέσματα συνενώνονται σε ένα σύνθετο hash key. Δύο διανύσματα θεωρούνται υποψήφιο ζευγάρι (candidate) αν συμπέσουν στο ίδιο bucket έστω σε ένα από τα L tables. Η επιλογή των παραμέτρων L, k και w καθορίζει τη σχέση μεταξύ ανάκλησης και υπολογιστικού κόστους:

- Μεγαλύτερο L : υψηλότερο recall, περισσότερα υποψήφια ζεύγη (candidates), περισσότερος χρόνος εκτέλεσης
- Μεγαλύτερο k : λιγότερα false positives, χαμηλότερο recall
- Μεγαλύτερο w : πιο μεγάλα buckets, περισσότερα υποψήφια ζεύγη, υηλότερο recall

Για τον σκοπό της παράλληλης εκτέλεσης, χρησιμοποιείται το Apache Spark [3], ένα framework παράλληλης επεξεργασίας, το οποίο κατανέμει τα δεδομένα σε partitions και τα επεξεργάζεται παράλληλα. Ως παραδοχή, αξίζει να σημειωθεί ότι το περιβάλλον στο οποίο εκτελέστηκε η εργασία, είναι το Google Colab, χρησιμοποιώντας PySpark σε local node, καθώς δεν κατέστη εφικτή η πρόσβαση σε κατανεμημένο σύστημα. Ωστόσο, μπορεί να ειπωθεί ότι η συγκεκριμένη υλοποίηση προσομοιώνει την παράλληλη εκτέλεση. Συνεπώς, ο παραλληλισμός περιορίζεται στους διαθέσιμους πυρήνες του μονού κόμβου, γεγονός που επηρεάζει τους χρόνους εκτέλεσης που παρουσιάζονται στην πειραματική αποτίμηση.

Το σύνολο δεδομένων το οποίο χρησιμοποιείται είναι το SIFT1M [4], ένα ευρέως διαδεδομένο dataset, ως benchmark, που αποτελείται από 1.000.000 διανύσματα, 128 διαστάσεων.

Περιγραφή αλγορίθμου

Σε αυτό το κεφάλαιο, περιγράφεται σε βήματα η λειτουργία του αλγόριθμου.

1) Φόρτωση δεδομένων και δημιουργία των Spark DataFrames. Μορφή κατάλληλη για partitioning.
2) Δημιουργία Hash Functions. Δημιουργία των τυχαίων διανυσμάτων a , b , εκτός Spark αρχικά. Στην συνέχεια γίνεται broadcast σε όλους τους workers. Αποτελούν παραμέτρους της Hash Function.
3) Υπολογισμός των Hash Keys: Αρχικά, ορίζεται μια User Defined Funtion (UDF), που εφαρμόζεται παράλληλα σε κάθε διάνυσμα. Για κάθε table l και κάθε hash function j υπολογίζεται:
$$h(x) = \lfloor (a \cdot x + b)/w \rfloor$$
4) Τα k αποτελέσματα κάθε table συνενώνονται σε ένα σύνθετο hash key (π.χ. "3,7,2"), το οποίο αντιστοιχεί στο bucket του διανύσματος, στο συγκεκριμένο table. Κάθε διάνυσμα παράγει L hash keys, δηλαδή ένα για κάθε table.
5) Αλλαγή format του DataFrame, σε μορφή (id, table,_id, hash_key)
6) Join για εύρεση των υποψηφιων: inner join μεταξύ των exploded DataFrames των R, S , με κλειδί (table_id, hash_id). Δύο διανύσματα θεωρούνται υποψήφια εάν πέσουν στο ίδιο bucket έστω σε ένα από τα tables. Η εκτέλεση του join πραγματοποιείται παράλληλα.
7) Επαλήθευση: για κάθε υποψήφιο ζεύγος, υπολογίζεται η Ευκλείδεια απόσταση μέσω UDF: $d(r,s) = \sqrt{(\Sigma(ri - si)^2)}$
8) Τέλος, κρατούνται μόνο τα ζευγάρια με $d(r,s) < \theta$

Πειράματα και αποτελέσματα

Για την εύρεση του βέλτιστου Recall, χρησιμοποιήθηκαν υποσύνολα των 2000×2000 διανυσμάτων, καθώς σε τέτοιο μέγεθος ήταν εφικτή η σύγκριση με τον baseline αλγόριθμο.

Επίδραση του *L*

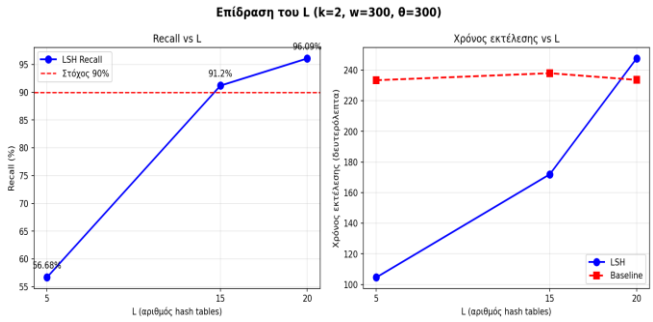
Το πρώτο πείραμα αφορούσε του αριθμού των hash tables στο recall και τον χρόνο εκτέλεσης, με σταθερές τις παραμέτρους και (=300).

Πίνακας 1: Επίδραση *L*

L	*Recall*	*LSH sec*	*Επιτάχυνση*
5	56.68%	104	2.2
15	91.20%	172	1.4
20	96.09%	247	0.9

Παρατηρείται ότι καθώς το L αυξάνεται από 5 σε 20, το recall αυξάνεται σημαντικά από 56.68% σε 96.09%. Ο στόχος recall >90% επιτυγχάνεται για L≥15. Ωστόσο, η αύξηση του L συνοδεύεται από αύξηση του χρόνου εκτέλεσης λόγω του μεγαλύτερου αριθμού υποψήφιων ζευγαριών. Το L=15 αποτελεί το βέλτιστο σημείο ισορροπίας, επιτυγχάνοντας ανάκληση 91.20% με επιτάχυνση 1.4x έναντι του baseline.

Εικόνα 1: Επίδραση L



Επίδραση του *w*

Στο δεύτερο πείραμα εξετάστηκε η επίδραση του πλάτους bucket w, με σταθερό k=3 (θ =300).

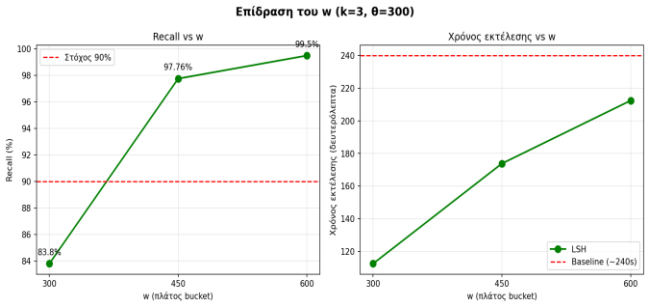
Πίνακας 2: Επίδραση *w*

L	*w*	*Recall*	*LSH sec*	*Επιτάχυνση*
30	300	83.80%	112	2.1
25	450	97.76%	174	1.4
20	600	99.5%	212	1.1

Παρατηρείται ότι το έχει καθοριστική επίδραση στο recall. Με =300, φτάνει στο 83.80%, ενώ με =450 και =600 ανεβαίνει στο 97.76% και 99.50% αντίστοιχα. Αυτό επιβεβαιώνει ότι το πρέπει να επιλέγεται μεγαλύτερο ή ίσο του , ώστε τα buckets να είναι

αρκετά "μεγάλα" για να συμπεριλάβουν τα περισσότερα κοντινά διανύσματα.

Εικόνα 2: Επίδραση w

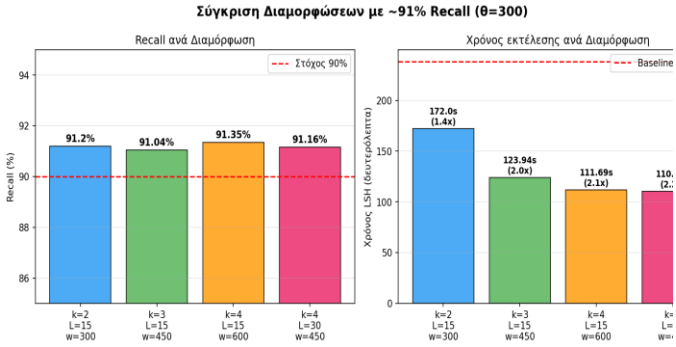


Πίνακας 3: Σύγκριση

k	*L*	*w*	*Recall*	*LSH sec*	*Επιτάχυνση*
2	15	300	91.20%	172	1.4
3	25	450	91.04%	124	2
4	20	600	91.35%	112	2.1
4	30	450	91.16%	110	2.2

Παρατηρείται ότι ενώ όλες οι αλλαγές παραμέτρων, επιτυγχάνουν παρόμοιας τάξης recall ~91%, ο χρόνος εκτέλεσης διαφέρει σημαντικά. Η αύξηση του k, συνδυασμένη με αντίστοιχη αύξηση του w, οδηγεί σε μικρότερα αλλά πιο αποτελεσματικά buckets άρα και σε λιγότερα candidates, γρηγορότερο verification. Η διαμόρφωση k =4, L =30, w =450 επιτυγχάνει ανάκληση 91.16% με επιτάχυνση 2.2×, βελτιώνοντας τον χρόνο κατά 36% σε σχέση με k =2.

Εικόνα 3: Σύγκριση



Όσον αφορά την κλιμακωσιμότητα, εφόσον ο baseline αλγόριθμος, σε δοκιμή 10000×10000 διανυσμάτων, δεν ολοκληρώθηκε σε διάστημα μεγαλύτερο των 2 ωρών, αποδεικνύεται το γεγονός ότι η εύρεση ακριβούς λύσης σε σύνολα δεδομένων τέτοιον διαστάσεων είναι πρακτικά αδύνατη και ασύμφορη. Αντίθετα, ο LSH αλγόριθμος με παραμέτρους k =4, L =30, w =450 εκτελείται σε ~110 δευτερόλεπτα για 2.000×2.000 διανύσματα, με αναμενόμενη υποδεκαπλάσια αύξηση χρόνου για μεγαλύτερα datasets λόγω του υπογραμμικού χαρακτήρα της κλιμάκωσής του.

Συμπεράσματα

Στην παρούσα εργασία, υλοποιήθηκε ένας αλγόριθμος, ο οποίος υλοποιεί προσεγγιστική διανυσματική σύζευξη και βασίζεται στον αλγόριθμο LSH. Χρησιμοποιήθηκε το Apache Spark, σε περιβάλλον Google Colab μέσω της pyspark. Τα πειράματα διενεργήθηκαν στο διανυσματικό σύνολο δεδομένων SIFT1M και οδηγούμαστε στα παρακάτω συμπεράσματα: Αρχικά, επιτυγχάνεται το επιθυμητό recall της τάξης >90%, με την κατάλληλη επιλογή παραμέτρων. Η βέλτιστη διαμόρφωση που προέκυψε από τα πειράματα είναι k=4, L=30, w=450, η οποία επιτυγχάνει ανάκληση 91.16% με χρόνο 2.2 φορές πιο γρήγορο έναντι αυτόν του baseline. Γενικότερα συμπεραίνουμε πως η επιλογή παραμέτρων είναι αυτή που παίζει τον πιο καθοριστικό ρόλο στην εξαγωγή ποιοτικών αποτελεσμάτων. Τέλος, είναι σημαντικό να αναφερθεί το πλεονέκτημα της κλιμακωσιμότητας του αλγορίθμου, για χρήση σε πολύ μεγάλα σύνολα δεδομένων όπως το SIFT1M.

Βιβλιογραφία

[1] Indyk, P. and Motwani, R. 1998. Approximate nearest neighbor: towards removing the curse of dimensionality. Proceedings of the Symposium on Theory of Computing.

[2] Datar, M., Immorlica, N., Indyk, P., and Mirrokni, V.S. 2004. Locality-Sensitive Hashing Scheme Based on p-Stable Distributions. Proceedings of the twentieth annual symposium on Computational geometry (SCG '04), 253–262. DOI: https://doi.org/10.1145/997817.997857

[3] Zaharia, M. et al. 2016. Apache Spark: A Unified Engine for Big Data Processing. Communications of the ACM 59, 11 (2016), 56–65. DOI: https://doi.org/10.1145/2934664

[4] Jégou, H., Douze, M., and Schmid, C. 2011. Product Quantization for Nearest Neighbor Search. IEEE Transactions on Pattern Analysis and Machine Intelligence 33, 1 (January 2011), 117–128. DOI: https://doi.org/10.1109/TPAMI.2010.57. Dataset URL: http://corpus-texmex.irisa.fr

[5] Rajaraman, A., Leskovec, J., and Ullman, J.D. 2014. Mining of Massive Datasets. Cambridge University Press. Available: http://www.mmds.org